

SIMATS ENGINEERING
ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1518
---------------------	--	---------------------	---------

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I DILLILOKESH D(192311097) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: DILLILOKESH

SIMATS ENGINEERING

CO2 ASSESSMENT TOOL 1

Introduction

This document walks through five practical business situations where an organisation runs into trouble because of the way its systems are built, and lays out a cloud-based fix for each one. For every case, the write-up covers what the organisation is dealing with, what could go wrong, how the new architecture is put together, and why that design is the right call.

Case 1 — Streaming Platform Struggling Under Peak Load

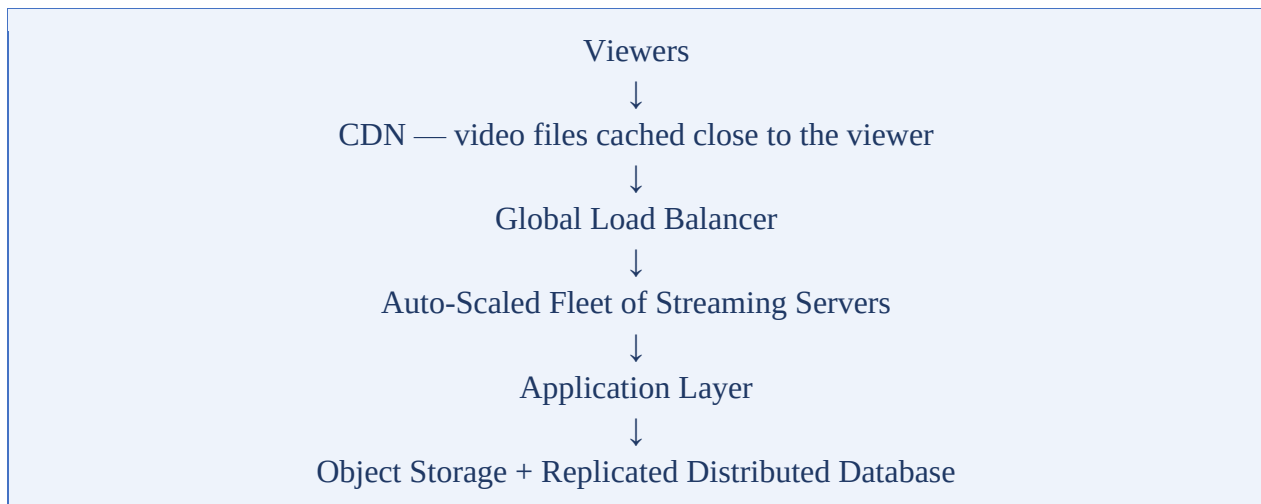
Situation

A media company wants to move its video streaming service to the cloud. The service needs to hold up under millions of simultaneous viewers, keep video playback smooth, and not let infrastructure spend spiral out of control during traffic spikes.

What Could Go Wrong

- Sudden viewer surges during live broadcasts overwhelming the servers
- Buffering and stalled playback when servers are overloaded
- Slow load times for viewers far from the origin servers
- Cloud bills climbing sharply during high-demand periods
- The whole service going down if one server or region fails

Architecture



How Traffic Is Managed

The load balancer spreads incoming requests across the healthy servers so no single machine gets overwhelmed, and it automatically stops sending traffic to any server that fails a health check. Underneath that, an auto-scaling group watches demand in real time — it spins up extra streaming servers when a live event draws a crowd, and quietly shuts down the unneeded ones once the crowd thins out, which keeps playback smooth without paying for idle capacity.

Keeping Costs in Check

- Serving cached video through the CDN cuts down on origin bandwidth charges
- Auto-scaling means the platform only pays for servers it's actually using
- Older, rarely-watched videos move to lower-cost archival storage
- Reserved instances cover the predictable baseline load
- Adaptive bitrate encoding shrinks the amount of data that has to move

Why This Design Makes Sense

Combining a CDN, a global load balancer, and auto-scaling gives viewers fast, uninterrupted playback no matter where they are, while the platform only pays for the compute it genuinely needs — a good balance of performance, uptime, and cost.

Case 2 — Fintech Startup Running Containers Across Clouds

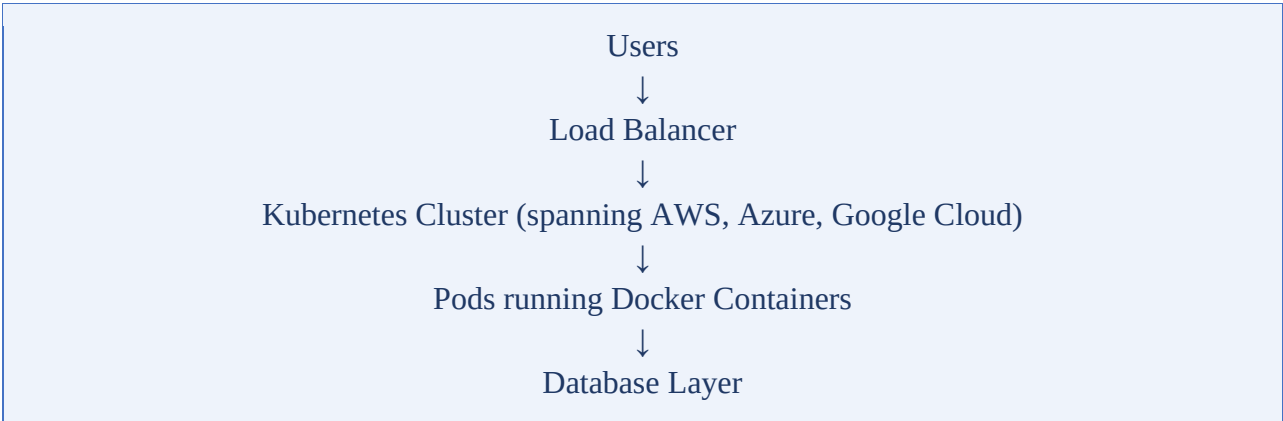
Situation

A fintech startup runs its containerised applications across more than one cloud provider. The difficulty is keeping deployments consistent, scaling smoothly, and managing everything without the process turning into chaos.

Risk	Why It Matters
Inconsistent deployments	Different environments on each cloud can drift apart and behave unpredictably
Vendor dependency	Leaning too heavily on one provider's tooling makes switching costly later
Network latency	Cross-cloud communication adds delay between services
Difficult monitoring	Tracking health and performance across separate platforms is harder
Security differences	Each provider has its own security model, which is easy to misconfigure

What Could Go Wrong

Architecture



Key building blocks: Docker containers, an Ingress Controller for routing, Istio as a service mesh, and a central monitoring layer that gives visibility across all three clouds at once.

How Services Find Each Other

Kubernetes Services handle discovery automatically, and internal DNS lets containers reach one another by name rather than hard-coded addresses, so pods can be rescheduled or scaled without breaking connections between services.

Scaling Behaviour

- Horizontal Pod Autoscaler adjusts the number of running pods based on load
- Cluster Autoscaler adds or removes underlying nodes as demand shifts
- Rolling updates deploy new versions without downtime

Trade-offs of Going Multi-Cloud

Benefits	Costs
High availability Built-in disaster recovery No single-vendor lock-in Wider geographic reach Pricing flexibility across providers	More operational complexity Higher running costs Data sync challenges between clouds Reconciling differing security policies

Why This Design Makes Sense

Kubernetes gives the startup one consistent way to deploy, scale, and manage workloads no matter which cloud they're running on, which makes the overall system more portable and resilient than tying everything to a single provider.

Case 3 — E-Learning Platform Recovering From Data Loss

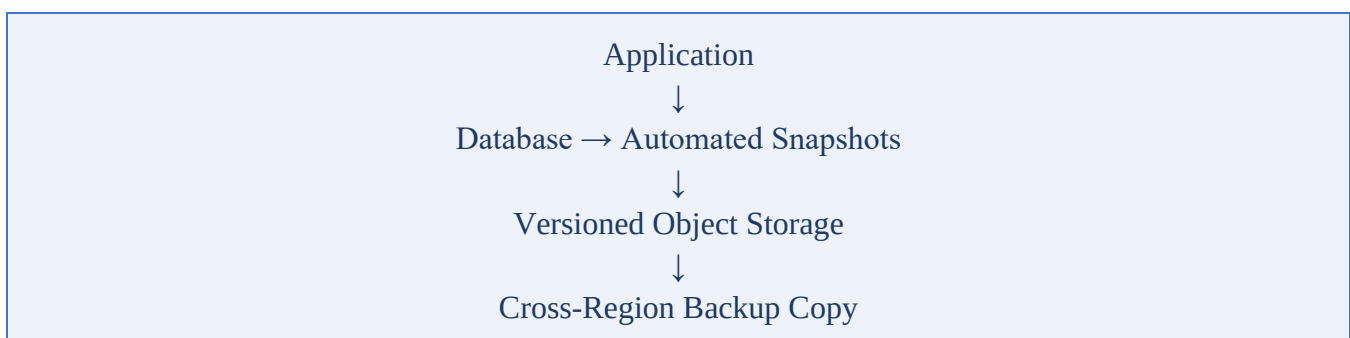
Situation

An e-learning platform suffered an accidental deletion of cloud data. It now needs a dependable backup and disaster recovery setup so this kind of incident doesn't happen again — or at least doesn't cause lasting damage.

What Could Go Wrong

- Data being lost permanently with no way to restore it
- Extended downtime while systems are repaired
- Students unable to access courses mid-session
- Database corruption spreading before it's caught
- The backup system itself failing when it's needed most

Backup Architecture



The setup layers several backup mechanisms: full backups once a day, incremental backups every hour, replication to a second region, versioning on stored objects, and automated snapshots on a schedule.

Recovery Targets

Metric	How It's Achieved
Recovery Time Objective (RTO)	Automated failover, standby servers on hand, and fast restoration from recent snapshots
Recovery Point Objective (RPO)	Continuous replication and frequent backups keep the amount of data at risk to a minimum

Steps to Keep the Platform Running

- Turn on automated backups by default
- Replicate backups into a second region
- Run disaster-recovery drills on a regular schedule
- Monitor backup jobs to confirm they actually succeeded
- Keep standby infrastructure ready to take over
- Write down the recovery procedure so anyone on the team can execute it

Why This Design Makes Sense

Layering versioning, cross-region replication, and automated backups together means the platform can recover quickly from almost any data-loss scenario while losing as little student data as possible.

Case 4 — Government Agency Balancing Security and Flexibility

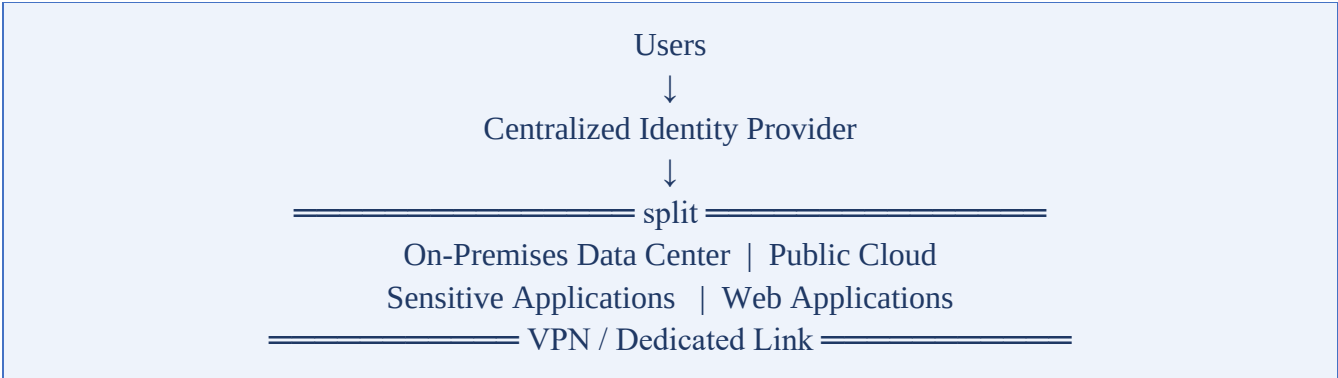
Situation

A government agency has to keep its sensitive systems on-premises for compliance reasons, while everything non-sensitive can live in the public cloud. The challenge is connecting the two halves securely.

What Could Go Wrong

- Data breaches exposing sensitive information
- Falling out of compliance with regulatory requirements
- Network-based attacks on either environment
- Identity and access management becoming overly complex
- Delays in communication between on-prem and cloud systems

Hybrid Architecture



Securing the Connection

- Site-to-site VPN between the two environments
- Dedicated private network connection where warranted
- Firewalls guarding both sides
- Network segmentation to limit blast radius

Managing Identity

- Single Sign-On (SSO) across both environments
- Multi-Factor Authentication (MFA) for all users
- Role-Based Access Control (RBAC) to limit what each user can touch
- One centralized identity provider as the source of truth

Protecting Data

- AES-256 encryption for data at rest
- TLS 1.3 for data moving across the network
- Encryption keys managed through a cloud KMS

Trade-offs to Accept

- More moving parts in the overall infrastructure
- Higher operational overhead than an all-cloud setup
- Some added latency between environments
- Ongoing compliance monitoring
- Keeping security policy aligned across both sides

Why This Design Makes Sense

Splitting workloads this way lets the agency keep sensitive systems under tight, compliant control on-premises while still getting the scalability and flexibility of the public cloud for everything else.

Case 5 — SaaS Provider Needing Zero-Downtime Resilience

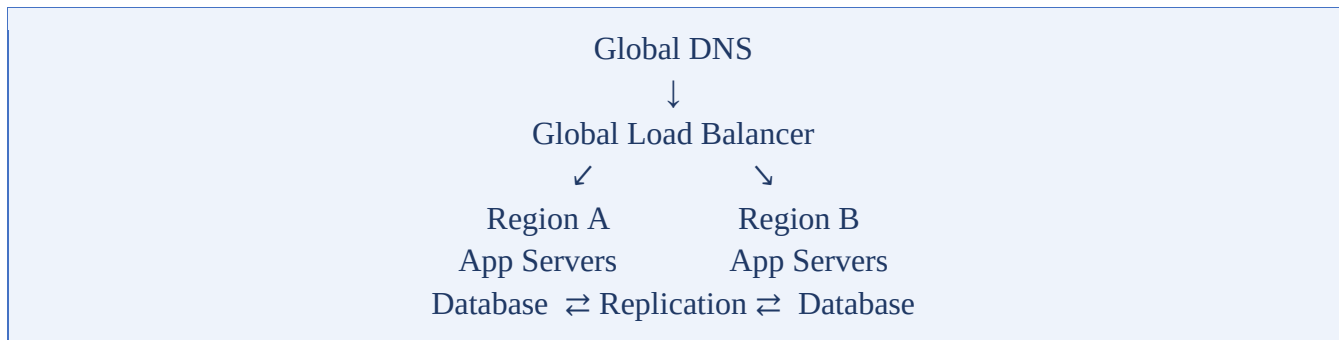
Situation

A SaaS provider needs its service to stay up even if an entire cloud region goes offline.

What Could Go Wrong

- A whole region going down unexpectedly
- Data loss during an outage
- Service interruptions for customers
- Hardware failures at the infrastructure level
- Network failures cutting off connectivity

Multi-Region Architecture



How Data Stays in Sync

- Synchronous replication for the databases that can't afford to lose a transaction
- Asynchronous replication for backup copies where a slight delay is acceptable
- Object storage replicated across regions as well

What Happens During a Failure

- Continuous health checks watch every region
- DNS automatically fails over to a healthy region
- The load balancer redirects traffic away from the failed region
- A warm standby environment is ready to pick up load immediately

Keeping an Eye on Things

- A central monitoring dashboard for the whole platform
- CPU and memory tracked continuously
- Application logs collected centrally
- Automated alerts via email and SMS
- AI-based anomaly detection flagging unusual behaviour early

Why This Design Makes Sense

Running active infrastructure in more than one region, keeping data replicated between them, and failing over automatically means customers barely notice even if an entire region fails — which is exactly the kind of resilience a SaaS provider needs.

Conclusion

Across these five cases, the same core cloud concepts keep showing up in different combinations — load balancing, auto-scaling, container orchestration, hybrid deployment, backup and disaster recovery, multi-region failover, caching, identity management, encryption, and monitoring. In each situation, the process was the same: identify what's actually at risk, design an architecture that addresses it directly, and explain why that architecture is the right trade-off between performance, security, availability, and cost.